Project Report

Mystery Vault

Uncover The Truth



Team Details:

Name	Register Number
Priyanka M	2025503044
Gopika M	2025503050
Srivarsha D M	2025503558
Tharnikaa Balakrishnan	2025503598

1.Problem Statement:

In today's digital world, people are often exposed to misinformation, misleading statements, and incomplete information. There is a need for an engaging way to develop critical thinking and evidence-based reasoning.

Mystery Vault addresses this through an interactive investigation game where players analyse evidence, question information, identify contradictions, and make decisions based on facts.

2.Software Requirement Specification (SRS):

2.1 Purpose and Scope

- **Product:** Mystery Vault — a single-player, standalone desktop detective game written in Java.
- **Purpose:** train evidence-based reasoning: collect evidence, interrogate suspects, spot contradictions, deduce the culprit.
- **User and scope:** one actor (the Player); four authored cases, 12 screens, local SQLite storage, no network features.

2.2 Functional Requirements

ID	Requirement	Screen / Module
FR1	Show a main menu and a detective profile with rank, XP and case statistics.	Screens 1–2
FR2	List the four cases with difficulty and completion status, and start the selected case.	Screen 3
FR3	Provide an Investigation Hub that links suspects, evidence, timeline, interrogation, connections, analysis and deduction.	Screen 4
FR4	Display suspect profiles (role, occupation, age, alibi, suspicion level) with View Details, View Evidence and Interrogate actions.	Screen 5
FR5	Catalogue evidence and unlock a clue only after its prerequisite evidence is discovered.	Screen 6 · ClueUnlockEngine
FR6	Run interrogations, record the dialogue and extract times and locations from statements using regular expressions.	Screen 7 · StatementRegexExtractor
FR7	Show a chronological timeline and flag conflicts between statements and facts.	Screen 8
FR8	Draw an interactive graph of relationships between suspects, evidence and clues.	Screen 9 · CustomGraphCanvas
FR9	Run evidence, timeline and statement analysis concurrently and report contradictions with severity.	Screen 10 · ConcurrentAnalysisManager
FR10	Accept suspect, motive and supporting evidence, check them against the sealed solution, and return a verdict and score.	Screen 11 · CaseSolverEngine
FR11	Award XP, update rank, mark the case solved and save everything to the database immediately.	Screen 12 · CaseCompletionService
FR12	Generate and export a case report (dossier) to disk.	ReportGeneratorService

2.3 Non-Functional requirements:

Category	Requirement
Usability	Noir navy-and-gold theme, custom Java2D components, consistent 12-screen navigation, keyboard shortcuts.
Data integrity	Foreign keys on, parameterised SQL, transactional case completion with rollback, no duplicate XP.
Isolation & security	One SQLite file per case; the canonical solution is never queried by player-facing screens.

Category	Requirement
Responsiveness	Analysis runs on a 4-thread pool; results return to the Swing Event Dispatch Thread.
Maintainability	Ports-and-adapters design: screens use ViewPort interfaces, never SQL directly.
Portability & testing	Java 17+, single runnable JAR (Maven Shade); 51 JUnit 5 test cases in 11 test classes.

2.4 Hardware and Software requirements:

Item	Requirement
Software	Windows 10/11 for the PowerShell/BAT scripts (JAR runs on any OS); JDK 17+; Apache Maven 3.8+; SQLite JDBC 3.45.1.0 (bundled in the JAR).
Hardware	Any standard desktop or laptop that runs JDK 17; display of 1440 × 900 or larger recommended.

3.Tools, Concepts and Software Used

3.1 Tools and Software

Category	Tool / Software	Purpose in the project
GUI (mandatory)	Java Swing, AWT, Java2D	Screens, CardLayout navigation, custom-drawn buttons, cards and graph canvas.
Database (mandatory)	SQLite 3 + sqlite-jdbc 3.45.1.0	Catalog database plus one isolated database per case, accessed with JDBC.
Language & build	Java 17, Apache Maven, Maven Shade	Compilation, dependencies and a single runnable JAR.
Testing	JUnit 5 (5.10.2)	Automated tests for services, database, analysers and UI.
Version control & scripts	Git, GitHub, PowerShell / BAT	Source hosting; build.ps1, run.ps1 and run.bat for one-step build and launch.
Diagramming	draw.io (diagrams.net)	Data flow diagrams and the use case diagram.

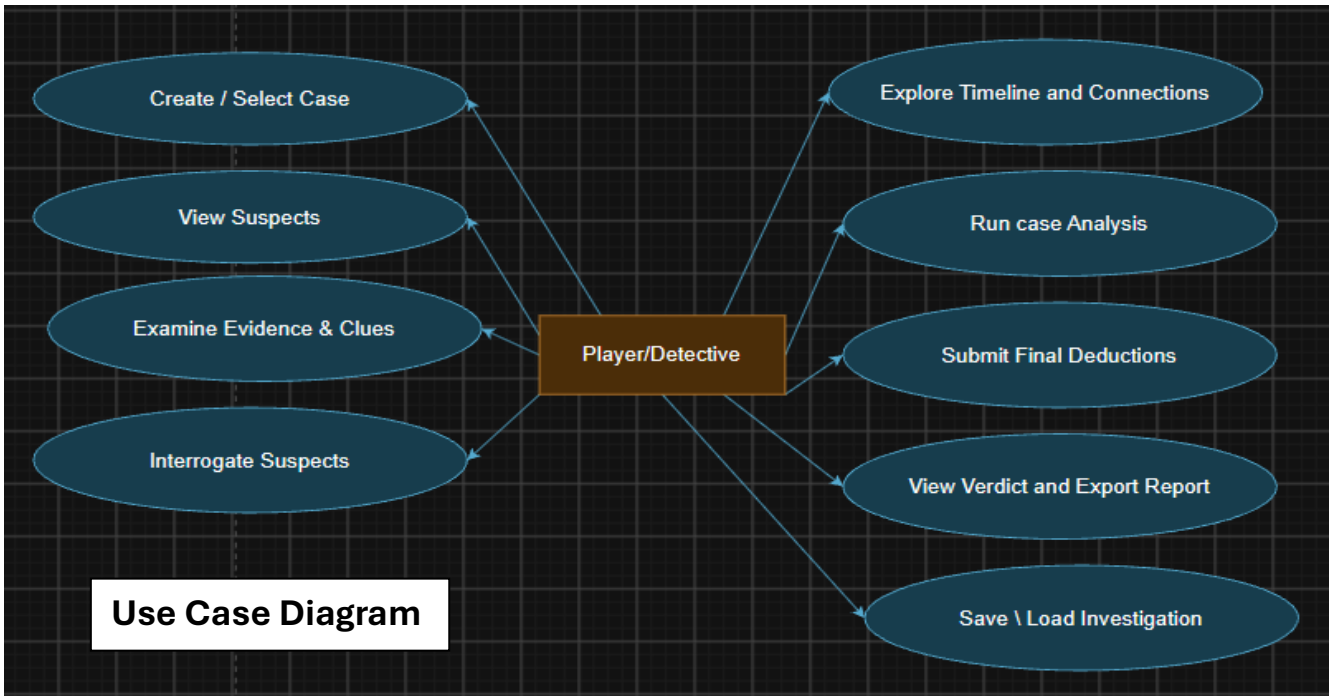
3.2 Java Concepts Used

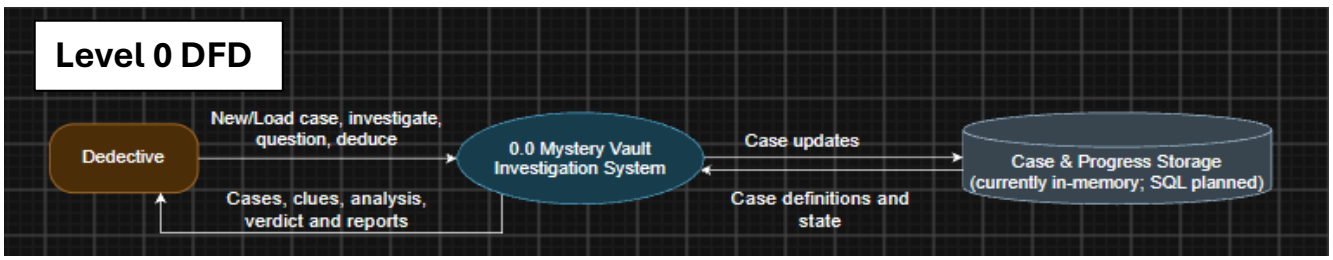
Concept area	Where it is used
OOP	Encapsulated view models and DTOs; inheritance and polymorphism in Swing panels and adapters; 10 ViewPort interfaces; enums such as DetectiveRank.
Core Java & exceptions	Strings, packages, static / final constants, java.time API; try-with-resources and SQLException handling with rollback.
Multithreading	ExecutorService and Future (analysis), synchronized (database manager), SwingUtilities.invokeLater (UI updates).
Collections	List, Set and Map with generics, lambda expressions and streams.
File handling & Regex	FileWriter for report export, Serializable DTOs, java.util.regex to extract times, dates and places from statements.
JDBC, GUI & events	PreparedStatement, ResultSet, transactions; ActionListener / mouse events, CardLayout, Graphics2D painting.

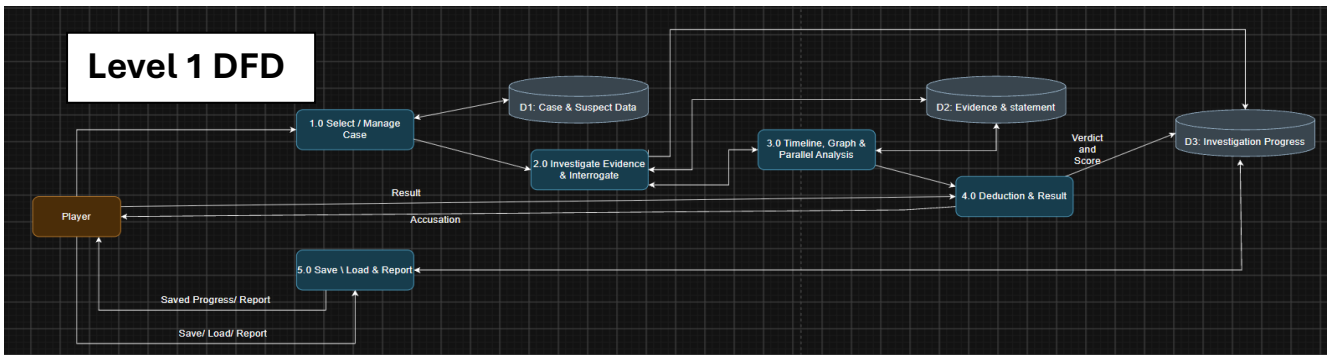
4.Highlights Of the Project

01 12-Screen Guided Workflow Menu → Profile → Cases → Hub → Suspects, Evidence, Interrogation, Timeline, Connections, Analysis → Deduction → Result.	**02 Four Authored Cases** Missing Diamond, Library Code, Poisoned Drink and Vanishing Prototype, each with five suspects.
03 Case Connection Engine Interactive Java2D graph linking suspects, evidence and clues.	**04 Contradiction Detection** Statements checked against evidence and timeline; severity LOW to CRITICAL.
05 Multithreaded Analysis Evidence, timeline and statement analysers run in parallel on a 4-thread pool.	**06 Strict Deduction Engine** SOLVED only if culprit, motive and decisive evidence all match; partial answers capped at 70.
07 Detective Progression Five ranks, Rookie to Master, at 0 / 100 / 300 / 600 / 1000 XP, saved between sessions.	**08 Isolated Databases** One SQLite file per case plus a catalog database for the player profile.
09 Clean Architecture Ports-and-adapters keeps the Swing UI independent of services and SQL.	**10 Automated Testing** 51 JUnit 5 test cases across database, solver, analysers, report and UI.

5. Use Case Diagram and Data Flow Diagram (DFD)







Link: https://drive.google.com/file/d/1kWMbewZk5qP599OxXvOcQ8U-sZaDfgSr/view?usp=sharing

6. Database Description

Table	Purpose	Relationship
Catalog database · mysteryvault_catalog.db		
cases_catalog	Case title, difficulty, database file name, unlocked / completed flags.	db_filename → the case's database file
player_profile	Detective name, XP, solved cases, correct / total deductions, best score.	Updated when a case is solved
saved_investigations	Saved progress percentage and notes.	FK → cases_catalog.case_id
Case database · case_101.db … case_104.db (one per case)		
suspects	Role, occupation, age, alibi, suspicion level.	Parent of the tables below
evidence	Type, location, significance, discovered flag.	associated_person_id → suspects
clues	Title, status (LOCKED / UNLOCKED / ANALYZED).	prerequisite_evidence_ids → evidence
timeline_events	Ordered events with conflict flag.	person_id → suspects
interrogation_dialogues, statements	Q&A and statements with extracted times and places.	suspect_id / person_id → suspects
contradictions	Title, severity, resolved flag.	Suspect, evidence, timeline IDs
connections	Graph edges: source, target, type, weight.	Any two entity IDs
canonical_solution	Sealed culprit, motive, decisive evidence.	Read only by solver and report
case_metadata, deduction_attempts, investigation_state, investigation_notes	Case status, attempts with score, state, notes.	Written during play

Relationships and Access

- **Relationships:** Databases are linked using foreign keys and consistent IDs.
- **Access:** JDBC and PreparedStatements manage database operations through services, keeping SQL separate from the UI.
- **Writes:** Correct deductions update case progress and player XP through coordinated transactions, with rollback on failure.

Git Hub Link: https://github.com/Tharnikaa/Mystery-vault.git